

Vision Day1 HW1 레포트

로봇 예비단원 주호진

1. AI 보고서

<https://share.gemini.google/74rVgMmj68PF>

openCV 카메라 영상을 topic으로 넘기려면
sensor_msgs/msg/Image로 넘기면 되나?

네, 맞습니다! OpenCV의 카메라 영상(이미지)을 ROS 또는 ROS 2 토픽으로 발행(Publish)할 때
`sensor_msgs/msg/Image` 메시지로 변환하려면 직접 필드를 채우기보다 ROS에서 공식 제공하는
`cv_bridge` 라이브러리를 사용하는 것이 가장 편리하고 안전합니다.

카메라 영상을 바로 띄우지 않고 ui node 에서 받아서 띄워야 했기에 메세지 타입에 대한 정보를 물어봤다.

cv bridge c++ 사용법

C++ 환경(ROS 2 기준)에서 `cv_bridge` 를 사용해 OpenCV의 `cv::Mat` 과 ROS의
`sensor_msgs/msg/Image` 를 상호 변환하는 방법입니다.

Cv_bridge 라는 라이브러리를 사용해서 cv::Mat 형식을 Image 로 변환하는 게 일반적이라고 했다.

```
cv_bridge::CvImage(header, encoding, frame).toImageMsg();
```

이런식으로 사용하면 된다는 걸 알았고, frame 은 cv::Mat 에서 자동으로 읽어준다고 한다.

```
/camera_node
/ui_node
/ui_node
/ui_node
/ui_node
/ui_node
/ui_node
/ui_node
/ui_node
/ui_node
```

ui node가 여러개인데 이유가 뭐야

`ros2 node list` 에 `/ui_node` 가 여러 개 출력되는 이유는 동일한 `ui_node` 실행 파일이 백그라운드나 다른 터미널에서 여러 번 실행되어 동시에 켜져 있기 때문입니다.

Bash

```
killall ui_node
```

(필요하다면 `camera_node` 도 함께 종료: `killall camera_node`)

2. 다시 확인하기

프로세스를 종료한 뒤, 다시 아래 명령어를 입력해 노드가 깔끔하게 하나만(또는 정상적으로) 뜨는지 확인해 보세요.

Bash

```
ros2 node list
```

중간에 `ui_node` 가 중복되어서 떠서 `killall` 을 치고 다시 확인하라는 답변을 받았는데 해당 명령어를 입력해도 사라지지 않았다가 모든 창을 다 끄고 다시 켜니까 사라졌다.

구독 받아온 걸 QObject 써서 qt에 싸주려하는데 어떻게 바꿔줘야 돼

ROS 2의 구독(Subscribe) 콜백 함수에서 받아온 이미지를 QObject 의 시그널(Signal)을 통해 Qt GUI로 쏴주려면, UiNode 가 QObject 를 상속받고 내부에서 emit 을 호출해주면 됩니다.

저번에 했던 qt 프로젝트에서 QNode 가 아닌 일반 node 를 연결해주는 구조를 사용해보고 싶어서 관련 구조에 대해 질문했다. public QObject 를 Node 보다 앞에 선언해준 다음에 signal 을 선언해주고,

```
auto sub_node = std::make_shared<SubNode>();
std::thread ros_thread([sub_node]() { rclcpp::spin(sub_node); });
ros_thread.detach();

MainWindow w(sub_node);
```

Qt main 함수에서 window 와 연결해준 다음, window 에서 signal 을 connect 해서 사용하는 방식으로 연결하면 됐다.

mutex 사용하는 이유

지금 qt아래에 구독노드가 있는데 mutex처리가 필요한거야?

1. Mutex를 사용하는 이유

Mutex (Mutual Exclusion, 상호 배제)는 멀티스레드 환경에서 여러 개의 스레드가 동일한 자원(변수, 메모리, 파일 등)에 동시에 접근해서 읽고 쓰는 것을 막기 위한 동기화 도구입니다.

- 만약 Mutex 없이 두 스레드가 동시에 같은 변수를 수정하려고 하면 데이터 경쟁(Race Condition)이 발생하여 값이 꼬이거나, 프로그램이 강제로 종료(Segmentation Fault)될 수 있습니다.
- 즉, "이 자원은 지금 내가 쓰고 있으니, 끝날 때까지 다른 스레드는 건드리지 마!" 하고 잠금(Lock)을 걸어주는 역할을 합니다.

구현에 mutex 가 있어서 질문하였다. 현재 구조에서는 굳이 mutex 를 적용할 필요가 없다고 해서 적용해야 하는 경우에 대해서 질문했다.

지금은 공유하는 게 아니라서 안 써도 된다?

네, 맞습니다! 정확하게 이해하셨습니다.

현재 구조에서는 다음과 같은 이유로 `mutex` 를 사용하지 않으셔도 안전합니다.

- **데이터를 공유하지 않고 복사(`clone`)해서 넘김:** ROS 콜백에서 받은 이미지를 그대로 쓰는 게 아니라, `.clone()` 을 통해 완전히 독립된 새로운 메모리 공간으로 복사한 뒤 Qt 시그널로 넘기고 있습니다.
- **스레드 간 직접 충돌 없음:** 두 스레드(ROS 백그라운드 스레드와 Qt 메인 스레드)가 동시에 같은 메모리나 변수를 직접 읽고 쓰는 상황이 없습니다.

만약 다른 곳에서 똑같은 이미지를 사용하고 이 이미지에 대한 정보가 변경될 가능성이 있을 때는 필요한건가



네, 맞습니다! 바로 그런 상황이라면 `Mutex`(동기화)가 필수적으로 필요합니다.

어떤 이미지 데이터를 여러 곳(다른 스레드, 다른 클래스 등)에서 동시에 공유하고 있고, 그 이미지의 내용이 계속해서 변경(갱신)될 가능성이 있다면 데이터 안정성을 위해 반드시 잠금 장치를 두어야 합니다.

`Mutex`가 필수적인 이유 (데이터 변경이 일어날 때)

- **Race Condition (데이터 경쟁) 위험:**
 - 예를 들어, A 스레드가 새로운 이미지로 데이터를 계속 덮어쓰고(수정하고) 있는 도중에, B 스레드가 그 이미지를 읽어가서 화면에 띄우거나 알고리즘 처리를 하려고 하면 메모리가 꼬이게 됩니다.
- **프로그램 비정상 종료:**
 - 메모리가 쓰여지는 도중에 읽어 가거나 동시에 접근하면 엉뚱한 주소를 참조하게 되어 `Segmentation Fault` 같은 에러와 함께 프로그램이 튕길 수 있습니다.

같은 데이터를 다른 곳에서 동시에 참고할 때 에러가 뜰 수 있는데, 이때 `mutex` 로 감싸준다면 한 곳이 작업 중일 때 다른 곳은 기다리게 만들어준다고 한다. 종괄호로 `mutex` 를 묶을 수 있고 종괄호를 탈출하면 풀리게 된다고 한다.